v1.0.0
20260215

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[Lab Notes]
PERFORMANCE OF CONCURRENT PROGRAMS

MULTITHREADING BENEFITS & COSTS

- Performance improvements...
  - improve *resource utilisation*
    - by letting applications more easily exploiting available resources and processing capabilities
  - improve *responsiveness*
    - by letting applications begin processing new tasks immediately while existing tasks are still running
- ...but also some new costs compared to single-threaded solution
  - overhead associated with coordinating between threads
    - locking, signaling, memory synch
  - increased context switching
  - thread creation and teardown
  - scheduling overhead

PERFORMANCE

- Improving performance = *doing more with fewer resources*
  - different kinds of resources
    - CPU cycle, memory, network bandwidth, I/O bandwidth, disk spaces
    - application resources: e.g. databases
  - programs bound to a resource
    - when performance of an activity is limited by the availability of a particular resource
    - e.g. CPU-bound, IO-bound, DBase-bound

MAIN ASPECTS

- **Throughput**
  - the rate at which a set of concurrent tasks is completed
- **Responsiveness**
  - the delay between a request for and completion of some action
  - also called *latency*
- **Scalability**
  - the improvement on throughput (or lack thereof) as more resources (usually CPUs) are made available

PERFORMANCE: TWOFOLD OBJECTIVE

- Distinguishing two different kinds of objectives
  - utilise the existing processing resources more effectively
  - enable our program to exploit additional processing resources if they become available
- ..that means (from the performance monitoring perspective)...
  - **keeping the CPUs as busy as possible (with useful work)**
  - for CPU-bound programs, increasing capacity by adding processors

PERFORMANCE TESTING

- Main goals of performance testing
  - seeking to measure end-to-end performance metrics for representative use cases
    - use-case selection is important
  - selecting sizings empirically for various bounds
    - number of threads, buffer capacities, etc.
- Two aspects
  - **testing throughput**
    - focus on *average*
    - answering to questions like: “how long does it take to an operation or task to be completed on the average ?”
  - **testing responsiveness**
    - focus on *variance*
    - answering to questions like: “what percentage of operations will succeed in under XX milliseconds?”

HOW FAST VS. HOW MUCH

- Application performance can be measured in a number of ways
  - service time, latency, throughput, efficiency, scalability, capacity
- *How fast* vs. *how much* aspects
  - ***how fast***
    - some measures how fast a given unit of work can be processed or acknowledged
      - service time, latency
  - ***how much***
    - some measures how much work can be performed with a given quantity of computing resources
      - capacity, throughput
- **Scalability** concept
  - describes the *ability to improve throughput or capacity when additional computing resources are added*
  - related to the *how much* aspect

PERFORMANCE VS. SCALABILITY

- Different goals
  - the goal when *tuning for performance* is do the same work with less effort
    - e.g. $O(n^2)$ to $O(n\log n)$
  - the goal when *tuning for scalability* is do more work with more resources
    - e.g. dealing with larger problem dimension by adding CPUs
- Two aspects of performance completely separate and sometime in conflict
  - how fast is typically of greater concern for desktop applications
  - how much (scalability, throughput, capacity) is typically of greater concern for server applications

PERFORMANCE TRADEOFFS

- Nearly all engineering decisions involve some form of tradeoff
  - example: quicksort more efficient for larger data set, bubble sort for smaller data set
- In any case: **avoid premature optimization** rule
  - first make it right, then make it fast (if it is not already fast enough)
  - don't trade safety for performance
    - the quest for performance is one of the more common sources of concurrency bugs
- Most premature are premature because they are often undertaken before a clear set of requirements is available

PERFORMANCE TESTING: MEASURING

- Before deciding that one approach is faster than another, we must clarify some aspects
  - what do you mean by faster?
  - under what conditions will this approach actually be faster? Under light or heavy load? With smaller or larger data sets? Can you support your answer with measurements?
  - how often are these conditions likely to arise in your situations? Can you support your answer with measurements?
  - is this code likely to be used in other situations where the conditions may be different?
- Importance of **measuring**
  - measure, don't guess.

EXAMPLE: MEASURING TIMINGS IN JAVA

- Exploiting functionalities provided by the virtual machine
- System class
  - `nanoTime()`
  - `currentTimeMillis()`

USING PROFILERS

- Profilers are tools that allow to evaluate the performance of program execution
  - monitoring thread state and use of the CPU
  - monitoring the time consumed by methods/procedures
  - …
- Examples for Java
  - VisualVM (free)
  - JProfiler
  - Java Mission Control

PERFORMANCE TESTING PITFALLS

- Watching out a number of coding pitfalls that can prevent performance tests from yielding meaningful results
  - **garbage collection**
    - longer tests to let GC run several times
  - **dynamic compilation**
    - again run the program for a long time so that compilation and interpreted execution represent a small fraction of the total run time
    - `-XX:+PrintCompilation` option to check when dynamic compilation runs
  - **unrealistic sampling of code paths**
    - test programs must adequately approximate the usage patterns of a typical application
    - *also*, test programs must adequately approximate *the set of code paths* used by such an application
  - **unrealistic degree of contention**
    - performance tests should try to approximate the thread-local computation done by a typical application in addition to the concurrent coordination under study
  - **dead code elimination**
    - avoiding the optimizer to optimize away our benchmark as dead code

BASIC GUIDELINES FOR OPTIMISING PERFORMANCE: HOW MANY THREADS

- Determining the ideal size for a thread pool
  - related to:
    - types of tasks that will be submitted
    - characteristics of the deployment system
- The size should be dynamically bounded to the number of available processors
  - so not hard-coded
  - Java example: using `Runtime.availableProcessors()`
- Empirical approach
  - considering the number of processors, the available memory, task type (CPU or I/O), task resource utilization
- Two basic kind of tasks
  - compute-intensive tasks
  - tasks that include also I/O or other blocking operations

RULE FOR THE COMPUTE-INTENSIVE TASK CASE

- A system with Ncpu processors usually achieves optimum utilization with a thread pool of **Ncpu + 1** threads
  - the (N+1)-th thread is useful since even in the compute-intense case threads can get page faults or pause for some reason, so the extra runnable thread prevent CPU cycles from going unused

RULE FOR TASKS WITH I/O OR BLOCKING OPERATIONS

- In this case we must set **larger** thread pool
  - since not all of the threads will be schedulable at all times
- How to size the pool - two ways:
  - running the application using several different pool sizes under a benchmark load and observing the level of CPU utilization
  - computing the optimal pool size for keeping the processors at the desired utilization by using the following formula:

`Nthreads = Ncpu*Ucpu*(1+W/C)`

where:

`Ncpu` = number of CPU
`Ucpu` = target CPU utilization (0 <= `Ucpu` <= 1)
`W/C` = ratio of wait time and compute time

- the ratio W/C of waiting time to compute time for tasks is estimated through profiling

HIDDEN SERIALIZATIONS

- All concurrent applications have some sources of *serialization*, heavily conditioning the speedup
  - strong impact on the speedup (see Amdahl's law)
- Often serialisation is hidden in frameworks and libraries
  - es: synchronized LinkedList vs. ConcurrentLinkedQueue in Java lib

MULTITHREADING COSTS: CONTEXT SWITCHES

- Context switch mechanism
  - saving the execution context of the currently running thread and restoring the execution context of the newly scheduled thread
  - manipulating shared data structures in the OS and the Language Virtual Machine (e.g. the JVM)
- Cost about 5000 to 10000 clock cycles
  - several microseconds on most current processes (2008)
- Tools for measuring how many context switches
  - `perfmon` on Windows
  - `vmstat` command on UNIX
  - `vm_stat` on Mac OS X

MULTITHREADING COSTS: MEMORY SYNCHRONISATION

- Distinguishing *contended* and *uncontended* synchronisation
  - if lock contention actually occurs or not
- Typically in VM uncontended synchronisation is fast
  - for instance, it can be handled entirely within the JVM
- Instead contended synchronisation may require OS activity
  - blocking control flows, scheduling, etc.
- The focus of optimisation efforts is on areas where lock contention actually occurs

REDUCING LOCK CONTENTION

- The principal threat to scalability in concurrent application is the **exclusive resource lock**
- Three ways to reduce lock contention
  - reduce the duration for which locks are held
    - narrowing lock scope
    - finer lock granularity
  - reduce the frequency with which locks are requested
  - replace exclusive locks with coordination mechanisms that permit greater concurrency
    - e.g. Read-Write locks
- Techniques
  - **lock splitting/striping** techniques
    - using separate locks to guard independent state variables

BIBLIOGRAPHY

- Goetz et al., “Java Concurrency in Practice”, Addison Wesley
  - Chapter 10, 11, 12
  - some examples have been adapted